事件驱动智能体操作系统跨层能力治理方法

徐刚[1,3]，冯骐[2,*]，姚俊杰[3]，肖宇[3]，陈铭松[3]，王江涛[1,3]，杨世光[3]，饶振宇[3]，王语欣[3]

[1]（华东师范大学国家可信嵌入式软件工程技术研究中心，上海）

[2]（华东师范大学信息化治理办公室，上海）

[3]（华东师范大学软件工程学院，上海）

通讯作者：冯骐，E-mail：qfeng@admin.ecnu.edu.cn

摘　要：事件驱动智能体操作系统将外部事件转化为任务并发起工具调用，事件来源、任务创建与工具执行分属不同信任域，需要建立贯穿执行链的授权依据。本文提出事件到智能体治理（Event-to-Agent Governance，E2AG）方法，以任务作用域能力绑定事件、任务与调用权限，在任务创建前和工具转发前分别实施授权检查，并通过关联执行记录复核授权依赖。来源–类型契约与目标策略决定任务准入，对象绑定的短期能力约束实际模型上下文协议（Model Context Protocol，MCP）调用。本文给出跨层授权状态模型及完整中介性质，并在事件驱动智能体操作系统原型中实现该方法。通过消融、外部机制对照、历史事件回放、异构配置实验及同路径测量，检验授权约束、执行证据及治理开销。实验中，完整方法保持正常业务可用并阻断指定未授权调用，移除任一执行检查点则出现相应违规调用到达上游的情况。提供相同授权上下文时，现有策略引擎和运行时规则获得一致的判定结果，支持将跨层对象关联和授权实施位置与具体判定机制分离。分阶段测量表明，治理增量主要集中于状态访问和证据持久化路径，其相对影响随业务计算量及并发条件变化。

关键词：人工智能操作系统；智能体操作系统；能力治理；完整中介；执行溯源

中图法分类号：TP311

A Cross-Layer Capability Governance Method for Event-Driven Agent Operating Systems

XU Gang[1,3], FENG Qi[2,*], YAO Junjie[3], XIAO Yu[3], CHEN Mingsong[3], WANG Jiangtao[1,3], YANG Shiguang[3], RAO Zhenyu[3], WANG Yuxin[3]

[1] (National Trusted Embedded Software Engineering Technology Research Center, East China Normal University, Shanghai, China)

[2] (Information Governance Office, East China Normal University, Shanghai, China)

[3] (School of Software Engineering, East China Normal University, Shanghai, China)

Abstract: Event-driven agent operating systems transform external events into tasks and tool invocations across distinct trust domains, requiring authorization that spans the execution chain. This paper presents Event-to-Agent Governance (E2AG), which binds events, tasks, and invocation permissions through task-scoped capabilities, enforces authorization before task creation and tool forwarding, and links execution records for authorization-dependency verification. Source–type contracts and target policies govern task admission, while object-bound, short-lived capabilities constrain actual Model Context Protocol (MCP) calls. We define a cross-layer authorization state model and a complete-mediation property, and implement E2AG in an agent operating system prototype. Ablations, external-mechanism comparisons, historical-event replay, heterogeneous-configuration experiments, and same-path measurements evaluate authorization constraints, execution evidence, and governance overhead. The complete method preserves normal workflows and blocks the specified unauthorized calls; removing either enforcement point allows corresponding violations to reach upstream tools. With the same authorization context, existing policy engines and runtime rules produce consistent decisions, supporting the separation of cross-layer object relationships and enforcement placement from the decision mechanism. Stage-level measurements show that additional processing time is concentrated in state access and evidence persistence, with relative overhead varying by workload computation and concurrency.

Key words: artificial intelligence operating system; agent operating system; capability governance; complete mediation; execution provenance

引言

本文将人工智能操作系统（Artificial Intelligence Operating System，AIOS）作为上位概念，指以人工智能（Artificial Intelligence，AI）为原生能力，为模型、智能体与系统资源提供统一运行支撑的系统软件范式；将其中面向智能体工作负载，统一管理模型、记忆、工具、生命周期、任务调度和访问控制的具体形态称为智能体操作系统（Agent Operating System，Agent OS）。AIOS 工作以代理应用与内核资源隔离为出发点，在内核中提供调度、上下文、记忆、存储和访问控制等服务 [1]；自主智能体综述则将画像、记忆、规划和行动概括为统一框架的主要模块，其中行动模块包括工具使用与环境交互 [2]。本文聚焦事件驱动 Agent OS：Git 推送、邮件、监控告警和业务回调不再只是被动输入，而可以直接创建智能体任务。任务运行时读取事件载荷，调用模型选择工具，再由 MCP 或其他连接器访问代码仓库、数据库、消息系统和设备。与传统请求–响应应用相比，这条链同时包含不可信事件输入、非确定性模型决策和可能改变外部状态的工具调用；治理问题因而不只是“模型选择了什么动作”，而是“该动作能否沿系统路径获得执行权”。

考虑如下贯穿全文的例子。一个多智能体博弈平台在参与者向公开测试代码库推送提交后，向本文 Agent OS 原型的隔离测试服务器发送标准化的 `git.push` 事件 e。服务器选择变更分析智能体 a 并创建任务 t；目标策略只允许该任务使用记录推送、记录评审和查询记录等工具，因此系统签发绑定 e, t, a 与允许集合 $U(g)$ 的短期能力 g。运行时产生的工具调用 c 若属于 $U(g)$，调用时策略执行点（policy enforcement point，PEP）才将其转发到上游；若模型选择固定的集合外状态修改工具，PEP 在接触上游前返回拒绝。该允许–拒绝配对对应本文自有设施上的治理一致性回归，但不用于估计自然流量中的模型违规概率。

同一例子给出两个单点机制无法覆盖的反事实分支。若来源–类型声明不满足接入契约但最终选择集合内工具，仅检查工具允许集合的 PEP 无法识别该违规，应在任务创建前完成来源准入；若来源与任务均已获准，但运行时选择集合外工具，调度前 PEP 尚未观察到实际调用 c，必须由调用时 PEP 再次验证。两条分支分别对应第 6 节的任务准入消融和上游调用可达性实验，说明问题不是重复增加规则，而是让授权依据随事件、任务和工具对象跨层传播。

CloudEvents、智能体间协议（Agent2Agent，A2A）和 MCP 分别规定事件、任务和工具调用对象 [16, 17, 18]，但协议互操作不自动产生跨对象授权。间接提示词注入评测揭示了不可信内容改变工具决策的风险 [6, 7, 8, 9]；AgentSpec 和 CaMeL 等工作把确定性约束置于模型之外 [10, 11]。将这些机制接入事件驱动系统时，还需明确三项相互依赖的授权：来源声明事件类型的权限、事件创建目标任务的权限，以及任务调用外部工具的权限。经典参考监控器要求安全相关访问受到不可绕过的完整中介 [20, 21]。在本文执行路径中，任务创建与工具转发是两个不同的受保护动作：前者需要在创建前检查事件准入，后者需要在实际请求形成后检查调用参数；二者可以共享策略判定器，但各自的检查必须先于对应动作。

本文把上述问题抽象为跨层授权传播：系统接收不可信事件后，应当显式建立后续对象的授权依据，而不是把事件到达或模型选择当作执行许可。在 Connector 已完成来源鉴别的前提下，事件只能声明契约允许的类型；获准事件只能创建目标策略允许的任务；任务只能使用绑定到其事件、智能体、MCP 服务和工具集合的短期能力。任何到达上游的工具调用，都应存在可验证的事件–任务–能力–调用依赖。该目标还要求审批、能力过期和事件重放采用单调、原子的状态转移，否则正确的策略判定仍可能被并发竞态破坏。

基于这一抽象，本文提出 E2AG 方法，把事件、任务、能力与调用组织为可检查的授权依赖，并在任务创建前与工具转发前实施检查。治理平面管理来源–类型契约、目标策略、审批和任务能力状态；执行平面承载两个 PEP；证据平面关联判定依据与执行记录。具体授权条件可以由现有策略引擎或运行时规则计算，跨层关系则决定判定器需要哪些状态、何时执行检查，以及事后如何核对其执行依据。

本文以状态迁移系统描述上述对象及其授权依赖，给出端到端调用授权性质与双执行点完整中介命题。实验分别检验执行检查点对授权约束的作用、对象关联与状态记录的可验证性，以及替换判定机制和部署接口后的行为一致性；同路径配对测量进一步量化实施这些机制的处理成本。

本文的主要贡献如下：

1. 提出事件驱动 Agent OS 的跨层授权问题，统一刻画事件声明权限、任务创建权限与工具调用权限，给出可由算法和实验共同检查的安全性质；
2. 设计 E2AG 三平面架构、任务作用域能力和双执行点完整中介算法，使工具调用不能脱离触发它的事件与任务单独获得权限；
3. 在事件驱动智能体操作系统原型中实现 E2AG，验证跨层授权及其证据关系，考察现有判定机制的接入与部署接口替换，并量化治理增量。

本文其余部分组织如下：第 1 节介绍相关协议、智能体运行时防护与访问控制基础；第 2 节给出系统模型、不可信输入能力和安全目标；第 3 节介绍 E2AG 总体架构；第 4 节给出跨层治理机制、算法与性质分析；第 5 节说明原型实现；第 6 节报告实验；第 7 节讨论适用边界与有效性威胁；第 8 节总结全文。

1 背景与相关工作

1.1 智能体风险与评测

间接提示词注入源于智能体把不可信数据同时解释为内容和指令，因而可能改变模型行为。InjecAgent 以工具集成任务评估间接注入，AgentDojo 提供包含任务、工具和注入情形的动态环境，ToolEmu 则以模型模拟工具执行结果以扩大长尾风险测试范围 [7, 8, 9]。现有中文综述分别从不同对象梳理相关风险与防护方法：黄河燕等将大语言模型风险归为模型自身安全与生成内容安全，并综述风险评估、归因和缓解方法 [3]；牟奕洋等从敌手目标、知识和能力出发，总结安全威胁与防御技术以及隐私威胁与保护技术 [4]；张熙等归纳大模型智能体的信息泄露、模型攻击、幻觉、伦理和法律合规风险及其防护建议 [5]。上述工作提供风险分类或模型行为评测，E2AG 则不判定自然语言意图类别，而是约束模型决策在真实系统执行链中的可达性。

1.2 智能体运行时防护

AgentSpec 通过领域专用规则描述触发条件、谓词和动作，在智能体运行时强制执行动作规则 [10]。CaMeL 从可信查询提取控制流和数据流，并用能力约束不可信数据对程序流与敏感数据外泄的影响 [11]。二者说明安全决策需要位于模型之外的确定性系统层。新近工作进一步把授权置于工具调用或智能体运行时：开放智能体通行证（Open Agent Passport，OAP）在调用前执行声明式策略并生成签名审计记录 [12]；ToolGuardian 结合工具准入表征与任务感知的声明式授权 [13]；持续智能体语义授权（Continuous Agent Semantic Authorization，CASA）将确定性消息约束与任务–工具语义匹配结合 [14]；Agent libOS 中的 libOS 指库操作系统（library operating system），该方法以显式能力中介进程、工具、资源和审批原语 [15]。这些方法共同证明了调用前授权的重要性，但其授权起点主要是用户任务、运行时进程或待调用工具。E2AG 进一步处理 Connector 外部事件的声明权，并把事件、系统任务和调用能力绑定为同一生命周期。参数谓词可直接约束一次请求中的字段值；跨步骤的数据流传播还需要 CaMeL 类机制，二者不能互相替代。

1.3 协议、参考监控器与能力系统

CloudEvents 规定事件格式及上下文属性，A2A 规定智能体发现、消息交互和任务生命周期，MCP 规定主机、客户端与服务器之间的工具发现和调用对象 [16, 17, 18]。这些规范提供各阶段的交互对象与安全要求。将它们连接为事件驱动执行链时，宿主仍需建立事件声明权、任务创建权与工具调用权之间的授权关系，并在相应动作执行前实施检查。OPA 将策略判定与策略实施解耦，使软件能够向外部策略引擎提交结构化输入并实施其判定 [19]；参考监控器和完整中介原则进一步要求安全相关访问始终经过不可绕过且可充分分析的检查机制 [20, 21]。早期能力系统以能力表表达主体对对象的访问权，Macaroons 则允许持有者通过增加 caveat 对派生凭据进行授权衰减和上下文约束，能力访问控制也可在操作系统内核中进行形式化验证 [22, 23, 24]。在决策能够执行之后，跨层复核还需要关联其对象依赖：万维网联盟（World Wide Web Consortium，W3C）的 Trace Context 支持跨服务传播跟踪标识和供应商状态；数字时间戳中的哈希链接使前后证书相互约束；in-toto 的布局和链接元数据则表达供应链步骤及产物依赖 [25, 26, 27]。这些机制分别支撑对象表示、授权实施与证据关联，E2AG 据此组织事件–任务–工具三层的授权传播和完整中介。

1.4 比较维度与研究定位

表 1 从状态提供、判定实施和证据关联三个方面比较已有机制与宿主的集成责任。通用判定器或运行时规则可以复用，来源鉴别、系统任务绑定和生命周期接入则由宿主或适配层承担。这一区分有助于确定比较条件，也为后续的策略引擎替换实验提供依据。

现有机制可以复用，关键在于补足其接入事件驱动系统所需的对象关联。例如，对引言中的代码推送任务，即使工具规则已允许写入评审记录，宿主仍需确认该任务由哪个获准事件创建、审批针对哪个对象，以及实际调用使用的能力是否仍有效。OPA 或 AgentSpec 可以计算相应授权条件，但必须由宿主提供这些状态并执行决定。Agent libOS 已提供进程、资源、工具及审批的能力中介；在本文场景中，还需把外部事件的来源与类型约束映射到进程创建依据，将系统任务与运行时能力关联，并保存跨对象的授权证据。E2AG 描述的正是这组跨层关系及其实施位置，而不是另造一种策略语言。

第 6 节据此安排不同层次的比较：OPA-Tool 检验工具边界输入的可见性，完整上下文 OPA 和共享准入宿主的 AgentSpec 检验同一约束的不同实施方式，Agent libOS 则比较共同的能力生命周期场景。前两种适配复用 DiOS 的对象管理，后一种使用 libOS 自身的进程与能力原语。三者是不同的实验配置，并非同时嵌入 E2AG 原型的组成模块。

表 1 代表性机制与事件驱动治理的集成责任

机制	可复用能力	接入本文运行路径时需明确的责任
CloudEvents	事件字段及上下文表示	Connector 来源鉴别、类型声明权限和任务准入
AgentSpec	触发器、谓词与运行时动作约束	宿主提供事件/任务状态，并将规则接入实际执行位置
OPA	结构化输入上的独立策略判定	宿主构造完整输入、实施决策并维护对象生命周期
CaMeL	控制/数据流隔离及能力约束	与事件准入、系统任务标识及调用出口建立映射
Agent libOS	进程、资源、工具与审批的能力中介	映射外部事件与系统任务，配置所需审计入口
E2AG	来源准入、任务能力与调用检查的组合	依赖受控入口/出口及可信治理状态，依赖验证须单独核验

2 系统模型与问题定义

2.1 系统对象与信任边界

本文研究如下系统：多个外部事件源经 Connector 接入 Agent OS，Agent OS 创建智能体任务，模型在任务中选择工具，工具调用可能改变外部资源状态。外部事件源及其载荷不可信；Agent OS 治理代码、策略库和持久化状态属于可信计算基；模型输出不作为授权依据；外部工具只信任经过调用 PEP 的请求。被保护资产是任务创建权限、工具凭据以及外部资源的完整性与可用性。不可信来源可能提交不一致事件字段、重复事件，或通过载荷改变模型工具选择；本文假设其不能直接修改治理代码、策略和数据库。目标是在不分析自然语言意图类别的前提下，使每个到达上游的工具调用都可追溯到合法事件、获准任务和有效任务能力。跨调用数据流、治理主机失陷和外部存储整体回滚不在本文范围内。

设来源主体集合、智能体集合和工具服务集合分别为 $\mathcal{S}$、$\mathcal{A}$ 和 $\mathcal{M}$。一次执行涉及事件 e、任务 t、任务能力 g 和工具调用 c。本文所称“层次”是授权对象在执行链中的语义层，而不是图 1 中组件职责的架构平面。定义授权层集合

$$\mathcal{H} = \{H_E, H_T, H_C\}, \qquad \lambda(e) = H_E, \quad \lambda(t) = H_T, \quad \lambda(c) = H_C, \tag{1}$$

其中 H_E、H_T 和 H_C 分别表示事件接入、任务创建和工具调用层；λ 给出对象所属层。“跨层治理”是前一层的授权结论成为后一层对象创建或使用的必要前提，而非三个平面之间的数据流。为刻画这些授权层之间的状态依赖，系统状态写为

$$\Sigma = \langle E, T, G, Q, L, V \rangle, \tag{2}$$

其中 E, T, G, Q, L 分别保存事件、任务、能力、审批状态和执行证据，V 保存调用意图及其结果观察状态。六类状态分别对应第 5 节原型中的 EventLog、A2ATask、ToolGrant、Approval、AuditEntry 和 ToolInvocation。

上述授权层描述对象语义；与之正交，系统部署还跨越事件接入域、智能体执行域和外部工具域，由此形成两个信任边界（trust boundary，TB）：TB1 位于外部来源与 Agent OS 之间，TB2 位于 Agent OS 与外部工具之间。跨越 TB1 的结构化字段不自动获得事件类型的声明权；跨越 TB2 的模型输出不自动获得工具执行权。

2.2 跨层授权状态迁移

令 $B_C(e)$ 表示事件来源与类型满足接入契约，$X_e = targets(e)$ 为该事件保存的候选智能体集合。目标策略判定采用事件级聚合，保存的准入判定记为 $D_P(e, X_e) \in \{\text{allow}, \text{deny}, \text{approval}\}$：任一目标的显式约束不满足，整个事件返回 deny；没有目标约束违规但命中敏感动作审批条件时返回 approval；其余返回 allow。该记号指准入时的结论，不表示调用时重新运行目标策略。来源鉴别是 B_C 的前置条件，字段模式匹配本身不构成身份认证。

审批对象绑定事件及其保存的目标集合，而非为每个目标分别建立审批。pending 状态经批准、拒绝或到期进入相应终态；批准后恢复该事件的任务分发。事件级准入只决定是否允许分发，不保证多个任务的创建、运行和恢复构成一个原子事务。对其中任一任务，一条正常路径表示为

$$\Sigma_0 \xrightarrow{\text{ingest}(e)} \Sigma_1 \xrightarrow{\text{admit}(e,X_e)} \Sigma_2 \xrightarrow{\text{create}(t)} \Sigma_3 \xrightarrow{\text{issue}(g)} \Sigma_4 \xrightarrow{\text{invoke}(c)} \Sigma_5 \xrightarrow{\text{intent}(c)} \Sigma_6 \xrightarrow{\text{forward}(c)} \Sigma_7. \tag{3}$$

其中 admit 完成事件至任务的授权传递，issue 建立后续调用依据，invoke 表示实际调用请求，intent 表示通过检查后先持久化调用意图。意图提交失败时不得转发；转发后若响应或结果持久化失败，调用保留结果未知状态。结果未知与治理拒绝不同，不能据此推断上游未执行。

2.3 端到端安全性质

本文将任务作用域能力定义为由可信治理平面签发、授予特定任务在给定期限内、按指定工具及参数约束发起调用的授权对象。它不是模型声明或工具名字符串，其使用必须通过记录中的任务绑定检查；其语义由签发依据、对象绑定和生命周期共同决定。令 $\text{Issued}(g,e,t,a,m,U,K,exp)$ 表示 g 由获准的 e,a 创建任务 t 后签发，并绑定服务 m、允许工具集合 U、参数约束 K 和期限 exp。$K_g(c)$ 检查实际请求的参数；未配置参数约束时取恒真，仅具有工具粒度语义。定义

$$\begin{aligned}\text{Admit}(e,a) \triangleq\ & B_C(e) \wedge a \in X_e \wedge \\ & \big(D_P(e,X_e) = \mathsf{allow} \ \vee\ (D_P(e,X_e) = \mathsf{approval} \wedge \text{Approved}(e))\big),\end{aligned} \tag{4}$$

$$\text{ValidGrant}(g,e,t,a,m,\tau) \triangleq \text{Issued}(g,e,t,a,m,U(g),K_g,exp(g)) \wedge \text{active}(g,\tau) \wedge \text{bind}(g,e,t,a,m). \tag{5}$$

其中 $\text{Approved}(e)$ 表示该事件的审批已获批准；τ 为 PEP 检查能力有效性的时刻，$\text{active}(g,\tau)$ 要求 $state(g) = \mathsf{active}$ 且 $\tau < exp(g)$；$\text{Forwarded}(c)$ 表示调用 c 已通过调用时 PEP 并被转发至上游工具端点。原型以可信状态存储中的能力记录、令牌摘要和 PEP 检查实现该语义，能力可信性依赖治理主机与存储的完整性。令 $e \prec t \prec g \prec c$ 表示可核对的授权前驱与对象关联，而非仅指时间先后。以下性质作为方法设计要求；其实现前提与已覆盖证据分别在第 4.6 节和第 6 节说明。

I1. 任务来源闭包：若创建 t，则存在 e,a 使 $\text{Admit}(e,a)$ 成立且 t 绑定 e,a；

I2. 能力作用域闭包：若签发 g，则它只绑定一个获准任务及其智能体、MCP 服务和允许工具集合及参数约束，且状态转移单调；

I3. 调用授权安全：若调用 c 已转发至上游，则必存在 e,t,g,a,m 使

$$\text{Forwarded}(c) \Rightarrow \text{Admit}(e,a) \wedge \text{ValidGrant}(g,e,t,a,m,\tau) \wedge \text{tool}(c) \in U(g) \wedge K_g(c) \wedge e \prec t \prec g \prec c; \tag{6}$$

I4. 证据连续性：对同一 trace 的可归属证据序列 $L_\rho = \langle l_0,\ldots,l_{n-1}\rangle$，序号连续、对象依赖满足状态迁移顺序，且非首项的前驱哈希等于上一项内容哈希。通过检查的调用在转发前具有持久意图，未取得可持久化响应时保留结果未知，不能补写成功。

据此，把治理目标定义为安全保持、可用性保持和可问责性三者的合取：

$$\text{GovOK}(\pi,\mathcal{W}) \triangleq \text{Sound}_{\text{I1:I3}}(\pi,\mathcal{W}) \wedge \text{Preserve}(\pi,\mathcal{W}_{allow}) \wedge \text{Accountable}_{\text{I4}}(\pi,\mathcal{W}), \tag{7}$$

其中策略配置 π 在工作负载 $\mathcal{W}$ 上必须满足 I1–I3；对契约、目标集合、工具及已配置参数约束均允许且基础设施正常的 $\mathcal{W}_{allow}$，不得由治理机制引入拒绝。可问责性要求对可鉴别、可关联的执行对象保存符合 I4 的证据；未知令牌请求不得写入他人的事件链，意图持久化失败时停止转发，已有意图但响应未能持久化时保留结果未知。式 (6) 给出中心安全约束，式 (7) 则避免以“全部拒绝”获得表面安全。第 6 节的 RQ1 与 RQ2 通过正常事件放行和未授权执行阻断检验安全保持与可用性保持，RQ3 通过证据关系核查和并发状态实验检验可问责性及状态单调性。其中，对象依赖须由关系验证器核对，不能仅由阶段定位和哈希连续性推出。

3 E2AG 总体架构

图 1 给出 E2AG 的静态系统架构。治理平面拥有授权决策及其持久状态，执行平面消费治理结果并实施完整中介，证据平面接收两个执行点和中间对象的状态变化。三个平面共享 trace 和对象标识，但职责不同：PDP 计算策略，PEP 阻止未获准状态迁移，证据平面记录一次决策依赖哪些对象。这里的“平面”回答组件由谁负责，“层次”回答授权约束哪个对象；一个平面可以同时服务多个授权层。

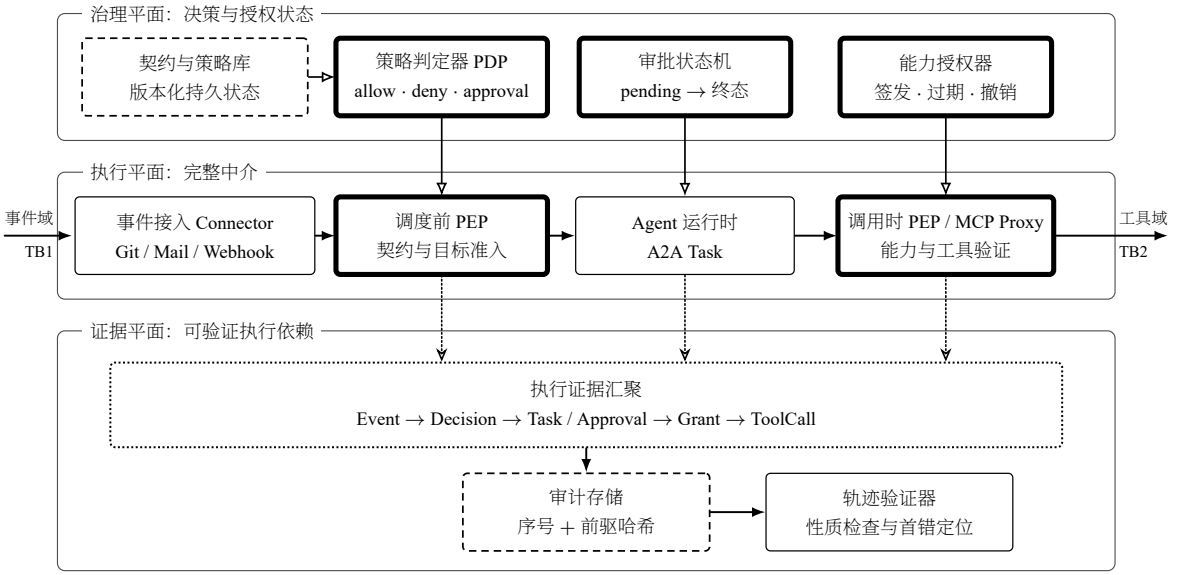


图 1　E2AG 总体架构。三个等宽平面按对象列对齐；实心箭头、空心三角箭头和点线空心燕尾箭头分别表示执行数据、治理控制和证据写入，双线框和虚线框分别表示 E2AG 强制组件和持久状态。

3.1　治理平面与授权状态

治理平面的设计动机是把授权依据从模型上下文中分离，并为跨层迁移提供唯一的可信状态源。它拥有版本化来源–类型契约、目标智能体策略、审批终态和任务能力生命周期；输入为标准化事件、目标与环境上下文，输出为 allow、deny 或 approval。能力参数由签发过程读取可信配置，不由模型给定。契约与策略库存储 B_C 和 D_P 的判定依据，治理判定器计算结果，审批管理器以条件更新完成一次性终态，任务能力管理器负责签发、过期和撤销。

治理状态不暴露给模型修改，模型只能消费与其任务绑定的能力，不能自行改变 $U(g)$、$exp(g)$ 或 $state(g)$。该约束为 I1、I2 提供持久状态基础，并在第 4.1–4.3 节分别落实为来源契约、目标策略和能力签发规则；第 5 节的 EventLog、Approval 与 ToolGrant 是这些状态的实现载体。

3.2　执行平面与完整中介

执行平面在任务创建和工具转发两个位置实施强制检查，使受控路径上的安全相关动作经过明确的授权入口。调度前 PEP 位于所有 Connector 汇合到任务创建的位置，观察事件来源、类型、目标智能体和环境上下文；其输出是拒绝、待审批或绑定事件的任务。调用时 PEP 位于 MCP 上游凭据之前，观察任务能力、调用时刻和模型最终选择的工具；其输出是本地拒绝或转发后的工具结果。

调度前 PEP 不能预测任务创建后的实际工具选择，调用时 PEP 也不能仅凭工具名恢复来源声明权，因此两个执行点并非重复检查。二者分别强制 $H_E \to H_T$ 和 $H_T \to H_C$ 的授权传递，共同保持 I1–I3；第 4.4 节算法给出其前置条件，RQ2 的双执行点配对实验通过移除任一执行点构造可观测反例。

3.3　证据平面

证据平面通过关联各阶段的授权对象，支持对治理结果的复核。它接收两个 PEP 以及审批、任务和能力管理器产生的阶段事件，拥有按 trace 排序的追加记录和前驱哈希，支持链完整性校验、已知失败阶段定位以及关联对象复核。拒绝和待审批路径记录已产生的事件与决策；获准路径继续关联任务、能力、调用和结果。

证据验证包括记录完整性与授权依赖正确性两类检查。前者核对序号和哈希，后者核对前驱、对象标识和状态迁移。第 4.5 节据此给出关系验证接口，RQ3 分别通过阶段故障、链内篡改和对象关系反例检验这些检查。

4 跨层能力治理机制

4.1 来源–类型能力契约

事件模式验证只回答字段是否合法。E2AG 的 Connector 描述包含来源模式集合，并把它与 Connector 可声明的事件类型绑定。契约判定先检查基本 CloudEvent 结构，再查找同时满足来源与类型的版本化契约。未找到绑定时默认拒绝，因此结构合法的 `source=webhook/attacker,type=git.push` 不能借用 Git Connector 的事件语义；这对应引言贯穿示例中“伪造来源但选择集合内工具”的第一条反事实分支。契约输出包含匹配类型、稳定原因码和判定证据，成为目标策略的输入。来源真实性依赖 Connector 的签名或身份验证；允许空验证密钥的开发配置不属于来源可信性结论的部署前提。

该机制拥有的是 H_E 的事件声明权，而不是目标任务或工具的最终执行权。对事件 e，只有 $B_C(e)$ 成立才进入目标解析；拒绝结果直接形成终态证据，不创建 t 或 g。因此来源–类型契约保持 I1 的第一个前置条件，并在 RQ1 的 ContractGuard 配置中与目标策略独立开关，以检验二者是否覆盖不同违规对象。

4.2 目标策略与审批状态

系统先取得候选集合 X_e 中目标的治理配置，检查事件来源、类型、声明动作和请求工具。任一目标存在显式约束违规时，事件整体拒绝；否则再检查生产环境中的敏感动作条件，决定进入审批还是直接分发。一次事件据此获得统一的准入结果，批准对象与后续分发的目标集合保持一致。

approval 不等价于临时放行。审批记录绑定事件，批准/拒绝操作以 pending 为条件更新，批准成功后恢复保存的目标集合；并发实验检验批准与拒绝竞争的单终态性质。任务分发与审批提交并非同一事务，因而该结果不构成任务恰好一次创建的保证。审批恢复也不会重新计算目标策略，能力签发则读取签发时配置；若准入、审批和签发之间允许配置变化，必须另行规定快照或重新授权规则。本文的授权继承论证限于该期间相关治理配置稳定的情形。

4.3 任务作用域能力

事件获准后，系统向保存的目标集合分发任务。任务运行时为每个受控工具服务签发能力

$$g = \langle \rho, id(e), id(t), id(a), id(m), U, K, exp, state \rangle. \tag{8}$$

其中 ρ 为 trace 标识，$id(\cdot)$ 为对象标识，U 是签发时工具名模式所表示的允许集合，K 是独立复制的参数约束，exp 和 $state$ 分别表示期限和生命周期状态。服务来自目标的可信配置，期限由配置的存活时长计算。策略判定产生事件准入结论，签发过程再将可信配置固化为任务能力，二者通过事件与任务标识关联。

参数检查以实际调用为输入，要求顶层字段集合与声明一致，并检查字符串枚举或长度界、整数区间及严格类型；不进行类型强制转换。精确资源标识可以作为字符串枚举约束，但这不等同于文件路径解析、信息流跟踪或报告内容正确性判断。K 的版本字段仅表示约束格式，未配置 K 时仅检查工具集合，显式但无效的约束不得静默退化为无限制参数。

调用时 PEP 检查令牌摘要、对象绑定、工具和参数约束以及能力有效性。Bearer 令牌关联到记录中的任务，但令牌本身不是独立的调用者身份认证；其保密性及上游凭据不向不可信运行时暴露是工程前提。任务终止或能力到期后，新请求将被拒绝；该语义不承诺撤回已通过检查的在途请求。

4.4 双执行点授权算法

图 2 描述单事件的准入和调用关系。算法 1 给出事件准入与获准事件分发，算法 2 给出工具调用过程，分别对应两个执行位置。伪代码聚焦治理步骤，省略去重和调度重试，但不将审批恢复、任务创建及能力签发合并为一个原子操作。

表 2 集中列出算法所依赖的记号。EvaluateContract 返回契约判定及原因，EvaluatePolicy 返回事件级三态判定；两者的决策字段用于分支，完整结果用于记录。HoldApproval 保存 pending 审批并返回，批准回调才调用 DispatchAdmitted。CreateTask 建立任务绑定，IssueConfigured 在任务运行时读取可信配置并签发零个或多个服务能力；无受支持服务时不签发。Append 表示阶段记录操作，不意味着所有调用共享一笔事务。

调用过程中的 CheckGrantAndCall 汇总令牌、能力状态、任务绑定、工具及参数检查，并确认上游服务存在；返回 deny 或判定故障时不转发。RecordIfAttributable 只为可鉴别且事件关联一致的请求记录拒绝，CommitIntent 在转发前提交意图并返回标识 j。ForwardOnce 尝试一次上游请求，CommitResponse 保存已观察到的响应及对应审计。后两步之间发生错误时返回结果未知，不自动重发；收到响应也

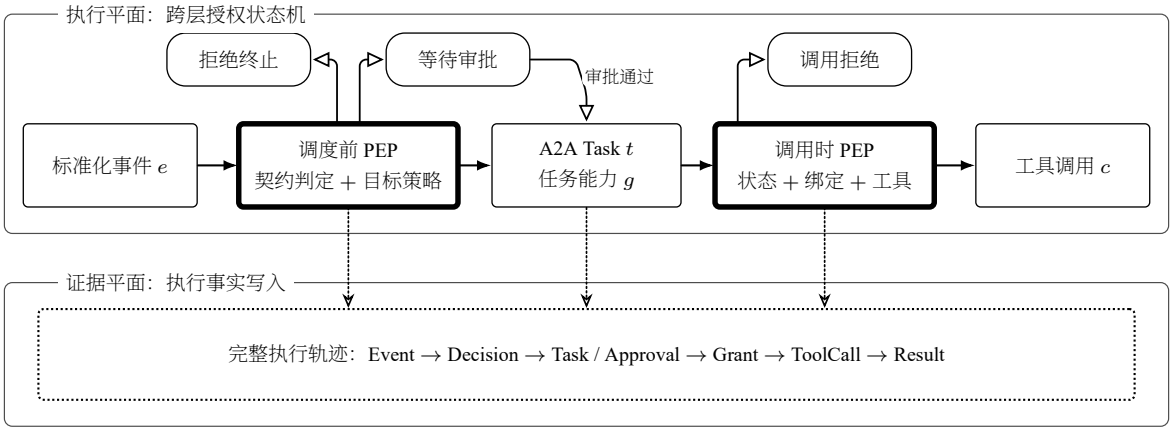


图 2　E2AG 跨层授权执行流程。两个等宽平面共享左右边界；实心箭头表示主执行路径，空心三角箭头表示治理分支，点线空心燕尾箭头表示证据写入，圆角胶囊表示暂停或终止状态。

表 2　核心记号及算法中的使用

记号	含义与使用位置
e, t, g, c; a, m	事件、任务、能力、实际调用；智能体与工具服务
$\mathcal{H}, \lambda$; Σ	三个授权层及所属映射；含调用观察状态的系统状态
ρ, X_e, $\mathcal{C}$, $\mathcal{P}$	trace、保存的候选目标集合、契约及目标策略配置
B_C, D_P, Approved	契约成立、事件级聚合判定、事件审批已批准
Admit, Issued	目标所属事件已获准；按可信配置为绑定任务签发能力
U, K_g, exp, $state, \tau$	允许工具集合、参数谓词、期限、能力状态及检查时刻
ValidGrant, $\prec$	能力有效性；授权前驱和对象关联，不仅是时间顺序
T_{new}, d_C, d, r, j	局部任务集合、契约决策、聚合决策、响应和调用意图标识

不等于业务完成。该过程针对经超文本传输协议（Hypertext Transfer Protocol，HTTP）POST 进入代理的工具请求；代理向上游转发时采用 Streamable HTTP 或受控标准输入/输出（standard input/output，stdio）管道。初始化等控制消息与工具业务调用分别记录。

算法 1 E2AG 事件准入与任务分发

procedure GovernEvent($e, \mathcal{C}, \mathcal{P}$)
  $\rho \leftarrow$ NewTrace; $X_e \leftarrow targets(e)$
  $d_C \leftarrow$ EvaluateContract($e, \mathcal{C}$)
  $d \leftarrow$ EvaluatePolicy($e, d_C, X_e, \mathcal{P}$)
  Append($\rho, \langle d_C, d, X_e \rangle$)
  if $d =$ deny **then**
    return $\emptyset$
  end if
  if $d =$ approval **then**
    HoldApproval(e, X_e, ρ); **return** $\emptyset$
  end if
  return DispatchAdmitted(e, X_e, ρ)
end procedure
procedure DispatchAdmitted(e, X_e, ρ)
  require 保存的直接允许判定，或该事件审批已批准
  $T_{new} \leftarrow \emptyset$
  for all $a \in X_e$ **do**
    $t \leftarrow$ CreateTask(e, a, ρ)
    任务运行时调用 IssueConfigured(e, t, a, ρ) 并记录签发结果
    Append($\rho, \langle t, a \rangle$); $T_{new} \leftarrow T_{new} \cup \{t\}$
  end for
  return T_{new}
end procedure

4.5 执行证据与验证

每项证据 l_i 包含 sequence、stage、outcome、对象标识、前驱哈希和内容哈希。令 H 为 256 位安全散列算法（Secure Hash Algorithm 256-bit，SHA-256），canon 为 JavaScript 对象表示法（JavaScript Object Notation，JSON）的确定性规范化函数，则

$$h_i = H\big(\text{canon}(l_i \setminus \{h_i\})\big), \qquad l_i.previous_hash = h_{i-1}. \tag{9}$$

记录从 $i = 0$ 开始，首项的前驱哈希为空字符串；式 (9) 的第二个等式适用于 $i > 0$。链校验器检查序号、trace、前驱哈希和内容哈希，阶段定位器检查已知 stage/outcome；二者均不单独证明对象绑定正确。为核对授权关系，离线验证器从同一一致性快照取得事件、任务、能力、审批、调用意图及其审计记录，按对象标识建立关联，并依次检查准入依据、任务归属、能力签发、调用意图和响应记录。缺少前驱、关联对象不匹配、重复意图或响应先于意图时，验证器返回相应错误，而不因哈希校验通过而接受。

依赖顺序与日志追加顺序并不完全相同。例如，任务已持久化后才签发能力，但任务分发完成的审计记录可以晚于能力签发记录；验证器因此核对任务实体与能力的绑定，不强制所有阶段名按图中的线性顺序排列。对于结果未知的调用，验证器保留该状态，不将其归为拒绝或成功。该实现核对强制治理模式下代理接收的 `tools/call` 记录关系，工具侧包括 HTTP 与受控 stdio 两种传输，依赖可信、一致的输入快照；由于快照不保存全部原始请求参数和历史策略，它不能重新计算全部授权判定，也不能证明上游业务已经完成。

算法 **2** E2AG 调用检查与结果记录

procedure AUTHORIZECALL(c, τ)
  $d \leftarrow$ CHECKGRANTANDCALL(c, τ)
  if $d \neq$ allow **then**
    RECORDIFATTRIBUTABLE(c, d); **return** d
  end if
  $j \leftarrow$ COMMITINTENT(c, outcome_unknown)
  if 意图提交失败 **then**
    return 错误（未转发）
  end if
  $r \leftarrow$ FORWARDONCE(c)
  if 未取得响应 **then**
    return 结果未知（j）
  end if
  if COMMITRESPONSE(j, r, response_received) 失败 **then**
    return 结果未知（j）
  end if
  return r
end procedure

哈希链检查已取得记录的修改、换序、插入和中间删除；整条链的尾截断检测还需外部 head/count 锚点。RQ3 因此分别检验阶段定位、链内完整性和对象关系，外部锚定及存储回滚问题留在第 7.3 节讨论。

4.6 安全性与复杂度分析

命题 **1**（受控路径上的调用授权）**.** 若（A1）本文范围内的事件驱动任务均经 GovernEvent 或其获准事件恢复分支创建，且 Connector 已完成来源鉴别；（A2）相应上游工具调用均经 AuthorizeCall，运行时不持有可绕过代理的凭据；（A3）治理代码和状态可信，准入至能力签发期间相关配置稳定，调用检查所读取的状态一致，则算法 1 和算法 2 在授权检查时刻 τ 的语义下保持式 (6)。

证明概要**.** 设调用 c 已转发。由 A2, 它通过能力、绑定、工具和参数检查且成功提交调用意图, 故在检查时刻有 ValidGrant(g, e, t, a, m, τ)、$tool(c) \in U(g)$ 及 $K_g(c)$。由 A1 和 A3, 任务来自保存的允许判定或该事件的获准审批, 配置未在准入至签发之间改变, 因此 Admit(e, a) 与 Issued$(g, e, t, a, m, U(g), K_g, exp(g))$ 成立，且建立 $e \prec t \prec g \prec c$。代入即可得到式 (6)。该推导不推出上游业务成功，也不证明证据验证器已覆盖 I4 的所有情况。 □

这些前提是工程约束而非由命题自动保证的事实。原型的入口检查和网络转发之间存在持久化步骤，不能将检查时刻有效解释为转发瞬间仍有效或即时撤回在途请求；审批之后重读配置也不提供决策快照保证。实验中的固定配置满足相关比较条件，但部署若允许并发改策略、撤销和动态换服务，需要明确相应生效时点并重新检验前提。第 6 节的消融只说明已实现路径上各检查的作用，不证明任何系统必须采用相同数量和位置的 PEP。

实现成本取决于实际数据结构。契约匹配遍历候选契约，目标策略遍历候选智能体及其约束；工具检查遍历允许模式，而不是对哈希集合执行常数时间查询。参数成本还随字段、枚举和字符串长度变化。审计追加复制和序列化已有链，处理量随链的序列化大小增长，并可能因条件更新冲突重试。因此调用检查与证据追加均不能统一视为常数时间操作，其端到端代价还取决于持久化和外部工具路径。

5 原型系统实现

本文在自主研发的开源事件驱动智能体操作系统 DiOS 上实现 E2AG。原型提供 Connector、CloudEvents 标准化、订阅匹配、事件调度、A2A Task 和 MCP 配置管理。E2AG 在此基础上集成契约与策略判定器、调度前 PEP、审批管理器、任务能力管理器、MCP 调用 PEP，以及证据记录和关系核查模块。各模块通过事件、任务和能力标识连接，表 3 列出具体接入路径。

所研究路径的调度前 PEP 位于 Connector 事件汇合到 A2ATask 创建的 dispatcher。EventLog 保存事件及契约判定，A2ATask 通过 context 和 trace 与事件关联，ToolGrant 保存式 (8) 的对象绑定、参数约束和生命周期状态；ToolInvocation 保存调用意图与响应观察状态。MCP PEP 先验证任务能力，再将获准请求转发至远程 Streamable HTTP 服务或受控 stdio 子进程；`tools/list` 结果按任务允许集合裁剪。stdio 服务须由管理员显式加入允许清单，可执行文件、参数和环境配置不交给任务；一个能力对应一个受应用进程管理的会话。未经启用的 stdio 服务以及服务器发送事件（Server-Sent Events，SSE）类传输不下发配置。

实现将纯判定与输入/输出（input/output，I/O）分离。纯函数负责契约和目标策略，dispatcher 负责对象创建与证据持久化，MCP PEP 负责调用转发前授权。实验因此可以在同一执行路径上独立开关契约、目标策略和两个 PEP，保持数据、模型输出和上游工具一致。

上述划分解决了判定位置和实验隔离问题，但并发状态仍可能破坏状态迁移的单调性。“先查询、后插入”的事件去重无法排除并发请求同时通过检查。原型采用带期限的唯一 EventDedupClaim，以数据库写入作为去重线性化点。审批使用 pending、approved、rejected、expired 四态和条件更新，使批准、拒绝及过期处理均仅能从 pending 进入终态。任务完成后的审计追加读取已提交的链并进行条件更新，避免旧会话缓存覆盖其他会话的证据；PostgreSQL 中审计链采用支持相等比较的 JSONB 类型。上述实现共同维护状态迁移的单调性与证据连续性。

为检验环境替换时的接口义务，实验还实现了签名 HTTP 推送、高级消息队列协议（Advanced Message Queuing Protocol，AMQP）0-9-1 消费和文件投递箱适配器。三者将交付标识与内容摘要映射到规范事件，完成持久认领后进入同一原生调度路径；来源分别绑定至可信端点、队列和目录配置。权威状态采用嵌入式 SQLite 或服务式 PostgreSQL，适配器的交付记录与原生事件、任务和审批对象关联。第 6.8 节比较这些配置的治理结果。

表 3 将命题前提落实到入口、出口与配置。A1 要求来源鉴别先于事件落库和任务准入；GitHub/Gitea 适配器检查请求体签名，GitLab 适配器检查共享令牌，两者不可视为同一种完整性保证。三个适配器均存在空密钥的开发旁路，故部署必须配置密钥并禁用该旁路，不能仅凭契约中的来源字符串声称已经完成认证。A2 要求受治理任务只获得任务代理配置，服务模式和直接调用接口需要另行隔离或接入治理。A3 则依赖可信配置和状态存储；能力参数快照可以防止后续配置放宽直接扩大既有能力，但不代替治理主机保护、配置生效时点管理或数据库一致性保证。

表 3 原型入口与出口的治理覆盖及部署义务

路径	治理方式	部署与保证边界
鉴别后 Webhook 至任务模式	经事件调度与任务准入	配置来源密钥；审批恢复沿同一事件目标集合
任务模式远程 MCP 调用	POST 工具调用经能力代理	运行时不得持有可直连上游的凭据
允许清单中的 stdio 服务	经能力代理管理会话并转发	可信管理员配置；单应用进程拥有管道会话
未启用 stdio 与 SSE 服务	强制模式下不下发配置	禁用不等于已验证全局出口拦截
直接 A2A 任务接口	未统一经过事件调度准入	限制入口访问或补充准入映射
服务模式与聊天调用	使用服务或聊天运行时配置	不纳入任务模式双 PEP 保证
Skill、本地进程与直接网络	未验证全局出口拦截	需操作系统或容器策略配合；不作已覆盖主张

6 实验设计与结果分析

6.1 研究问题

实验围绕以下三个研究问题（research question，RQ）展开：

- **RQ1**：来源契约和目标策略是否分别提供独立安全收益，组合后能否保持正常事件可用性？对应 I1；
- **RQ2**：在相同输入与工具端点下，准入和调用两个位置的检查分别约束哪些未授权执行，完整方法能否阻止相应用例到达真实工具端？对应 I2–I3；
- **RQ3**：执行证据能否定位治理失败并识别对象依赖不一致，去重与审批状态在并发下是否保持单调性？对应 I4 和式 (3)。

在上述安全与证据评价之外，第 6.7 节通过同路径对照量化启用治理后的处理成本，第 6.8 节检验不同接入、传输与状态存储配置下的治理一致性。

6.2 工作负载、规模与实验边界

实验分别检验来源准入与工具集合约束、参数约束、外部机制接入和执行证据。表 4 汇总工具粒度实验的配置与规模，包括来源准入、工具允许集合、阶段定位和状态竞争；参数约束、外部机制及调用意图关系的配置与结果在相应小节报告。60 例治理规则冻结矩阵由四名未参与语料构造的复核者通过盲表检查标签，复核不增加工作负载规模。700 次固定种子变异仅作为机制压力覆盖，其中序列化去重后有 319 个不同用例。确定性端到端重复用于检查状态稳定和配对因果，不增加工作负载多样性。模型决策回放预先冻结一个模型在三个固定场景中产生的 30 个工具选择结果；配对执行复用这些不可变结果，不由模型构造治理回归向量。另在自有设施上以公开测试仓库的真实推送构造允许路径正对照和固定集合外请求负对照，检查 ToolGrant 与运行时 PEP 的一致性；该配对轨迹不计入统计样本。

表 4 实验的独立维度与操作规模

维度	覆盖范围	证据边界
事件与规则情形	7 类来源；30 直接放行 +26 拒绝 +4 审批；7 类变异算子	四人盲表复核；变异集为合成压力数据
标签复核	4 名复核者 ×60 例	检查规则可复核性，不证明分布代表性
治理配置	Contract×Policy 4 种；双 PEP 4 种	内部消融，不等价于外部方法复现
外部策略引擎	OPA v1.17.0；49 例可调用切片 +8 例接口补充	隔离工具边界可见性，不报告本地时延排名
端到端路径	480 条确定性路径；90 条模型决策配对路径	重复验证稳定性，不代表独立样本规模
模型与提示	1 个模型；每场景 1 个固定提示	不估计开放环境中的模型违规选择概率
外部端点	loopback MCP canary；1 组公开仓库–自有设施配对回归	验证部署闭合与 grant–PEP 一致性，不构成统计工作负载
状态与并发	SQLite；8/32 并发；每配置 100 轮	不外推其他数据库后端

冻结矩阵包含 GitHub、GitLab、Gitea、互联网消息访问协议（Internet Message Access Protocol, IMAP）、通用 Webhook、Manual 和 Cron，共含 30 个应直接放行事件、26 个应拒绝事件和 4 个应转审批的生产敏感事件。冻结数据中的作者标签将后两类合并编码为 `attack`；盲评允许复核者独立判断事件类别。为避免把高风险但合规的审批事件等同于恶意攻击，本文以预期治理结果（放行、拒绝或审批）作为主要标签，并把事件类别一致性单独报告。所有样本满足基本 CloudEvent 结构，以避免把解析失败计为跨层治理收益。四名复核者仅接收移除作者标签的事件字段与目标治理声明，分别标注事件类别、首个治理层和完整 E2AG 预期决策。报告非直接放行事件处置率、正常通过率、Wilson 95% 置信区间以及多复核者 Fleiss κ。

6.3 基准方法与公平性

表 5 给出工具粒度实验的基准。内部配置共享代码路径、事件、模型决策和上游工具，仅切换治理位置。ContractGuard、DispatchGuard 和 RuntimeGuard 分别隔离事件声明绑定、前置策略门控和调用时工具约束，用于组件消融。OPA-Tool 使用官方 OPA v1.17.0 与 default-deny Rego 工具允许集策略 [19]，其输入限定为 MCP 方法、工具名和任务允许集合，用于考察工具边界可见的授权条件；事件来源、目标动作、审批和能力状态则由下述完整上下文配置提供。

为检验完整授权输入下策略判定器的可替换性，实验设置 E2AG-with-OPA 配置：Rego 从原始事件、目标权限、能力状态和请求参数独立计算决定，不接收 E2AG 预先计算的允许结果；审批、任务、能力和审计仍由同一宿主维护。AgentSpec 采用作者提交 `e6fa3902`

的解析器、解释器及原生受控执行器，按官方扩展接口注册业务谓词，并披露任务结束空值的兼容适配；48 项差分检查中仅 2 项出现预定的结束行为变化。Agent libOS 采用作者提交 `85e8ff94` 的进程、能力和 MCP 原语，比较双方均能表达的正常、过期与撤销场景。AgentSpec 的比较范围为共享准入宿主后的动作检查；Agent libOS 的比较范围为自身能力原语下的生命周期检查，其工具服务采用测试替身记录实际调用回执。两者分别检验规则实施和能力状态，完整的事件业务链仍由 DiOS 承载。

表 5　实验基准方法与治理能力

编号	名称	开启机制	隔离对象
C0P0 / G0R0	NoGuard	无跨层治理	无治理参照
C1P0	ContractGuard	来源–类型契约	事件声明权限
C0P1 / G1R0	DispatchGuard	目标策略/调度前 PEP	任务创建权限
G0R1	RuntimeGuard	任务工具允许集合	工具调用权限
OPA-T	OPA-Tool	OPA/Rego 工具允许集合	工具调用对象
C1P1 / G1R1	E2AG	准入与运行时能力	完整方法

6.4　RQ1：跨层任务准入

表 6　来源契约与目标策略的完整消融

配置	正确处置非直接放行	处置率	正常通过	正常通过率
C0P0	0/30	0.00%	30/30	100.00%
C1P0	10/30	33.33%	30/30	100.00%
C0P1	20/30	66.67%	30/30	100.00%
C1P1	30/30	100.00%	30/30	100.00%

表 6 显示 ContractGuard 只阻断来源–类型绑定违规，非直接放行事件处置率置信区间为 [19.23%, 51.22%]；当来源具有合法契约但目标智能体不允许其动作或工具时，契约不产生作用。DispatchGuard 阻断 16 个目标能力不匹配事件，并将 4 个生产敏感事件转入审批，区间为 [48.78%, 80.77%]，但无法识别来源绑定不匹配。完整组合覆盖两类互不重叠的治理原因，区间为 [88.65%, 100%]。四组正常通过率相同，说明差异来自治理对象而不是普遍收紧接入。

为确认上述预期决策并非仅由作者解释，四名未参与语料构造的复核者独立检查同一冻结矩阵。复核者对首个治理层和完整 E2AG 预期决策均达到 60/60 逐例全体一致，Fleiss κ 均为 1.00。事件类别有 56/60 例全体一致，原始一致度为 0.966667，Fleiss $\kappa = 0.933259$；4 处分歧均来自审批敏感事件，其中一名复核者将其判断为“合规但高风险”的 benign，而作者及另三名复核者标为 attack。该复核者仍对这 4 例的治理层 `approval` 和决策 `approval_required` 与其他复核者完全一致。四份表的冻结输入字段均未被修改，无缺失或非法值，且逐例备注在任意两名复核者间没有完全相同项。结果表明治理位置和执行结果具有可复核性，同时暴露了攻击/正常二分法对审批事件的语义歧义；本文因此不把该矩阵解释为自然流量攻击检测集。

在冻结矩阵之外，固定种子压力集进一步检查上述机制分工是否随输入变异而改变。压力集对 7 种变异算子各执行 100 次，NoGuard、ContractGuard 和完整 E2AG 的精确决策率分别为 0、57.14% 和 100%。ContractGuard 只覆盖必填字段、规范版本、来源–类型交叉和未知类型 4 类变异，完整方法继续覆盖需审批动作与目标能力不匹配。该结果验证实现对不同变异算子的稳定分工，但由于样本由规则生成，不用于估计真实分布中的检出率。

6.5　RQ2：双执行点与上游调用可达性

6.5.1　独立策略引擎的工具边界基线

从 60 例冻结矩阵中选择已经形成可比较工具授权对象的用例：目标非空、事件显式声明 `requested_tool`，且目标策略包含 `allowed_tools`。该规则得到 49 例可调用切片，包括 30 例正常和 19 例跨层违规；其余 11 例在工具对象形成前终止或未声明工具。若

把“没有工具对象”直接计作 OPA 拒绝，会虚增工具边界覆盖，故明确排除。OPA-Tool 只接收 `tools/call`、工具名和逐例任务允许集合；Rego 策略规定工具名匹配允许模式时通过，不调用 E2AG 判定代码。NoGuard 与 E2AG 行从同一冻结运行的 C0P0、C1P1 结果按这 49 个编号投影得到。为保持三态决策语义，正常例只有 `allow` 计为通过；违规例的 `deny` 或 `approval` 均计为“非允许”，表示该请求不会直接进入工具调用。

表 7　49 例可调用切片上的独立策略引擎基线

方法	正常通过	违规非允许	总体对齐
NoGuard	30/30	0/19	30/49
OPA-Tool v1.17.0	30/30	4/19	34/49
E2AG	30/30	19/19	49/49

表 7 显示 OPA-Tool 保持 30/30 正常调用，并对 19 例违规中的 4 例返回非允许。分层结果为：工具策略层 4/15 非允许，审批层 0/4 非允许；前 4 例均可由工具集合不匹配在调用边界直接观察。其余 15 例的工具名本身在允许集合中，违规信息位于来源、事件类型、目标动作或审批状态，工具边界输入不足以区分。完整 E2AG 通过调度前 PEP 补充这些上下文，因而在相同切片上得到 19/19 非允许。两种配置的差异在于授权输入：OPA-Tool 未取得的来源与审批信息由 E2AG 的准入路径提供。第 6.5.4–6.5.6 节进一步向 OPA 提供完整上下文，检验同一策略引擎在信息齐备时的授权行为。补充接口集中的 8 个 `tools/call` 请求上，OPA-Tool 与 RuntimeGuard 均正确处理 4 个集合内和 4 个集合外工具，证明工具允许集语义可由独立引擎复现；另 2 个非 `tools/call` 方法不计入该补充集。实验保存 OPA 发布件、策略、输入及投影结果的哈希，以便复核。

6.5.2　双执行点的确定性与模型决策配对执行

工具边界基线说明了输入可见性差异；进一步的问题是，该差异是否会转化为真实上游调用。为此，确定性实验包含正常授权、来源不匹配、任务后集合外请求和需审批动作，在 4 种配置下各重复 30 次，共 480 条 dispatcher–SQLite–A2ATask–ToolGrant–MCP PEP 路径。表 8 报告真实上游可达性。

表 8　确定性端到端执行中的上游调用可达性

配置	正常到达	来源不匹配	集合外请求	需审批动作
G0R0	30/30	30/30	30/30	30/30
G1R0	30/30	0/30	30/30	0/30（审批）
G0R1	30/30	30/30	0/30	0/30
G1R1	30/30	0/30	0/30	0/30（审批）

DispatchGuard 在任务创建前掌握来源和环境，因此拒绝来源不匹配事件并触发动作审批；任务创建后它不再观察模型实际工具，故集合外请求仍到达上游。RuntimeGuard 检查任务能力中的工具集合，阻断全部集合外请求，却不能判定一个获准工具是否由来源不匹配事件触发。完整 E2AG 保留正常调用并阻止三类未授权调用到达上游。该配对反例与命题 1 的两个完整中介假设一一对应。

确定性路径固定治理输入和预期上游到达结果，仍未覆盖既有模型工具选择进入同一执行路径后的系统行为。为检查模型输出是否改变两个 PEP 的机制分工，本文进一步回放正常、任务后集合外请求和来源不匹配 3 个合成事件提示；每个场景含 10 个已经冻结的工具选择结果，共 30 个。每个不可变结果在 G1R0、G0R1 和 G1R1 中复用，形成 90 条配对执行路径；回放过程不重新调用模型，上游为真实 loopback HTTP MCP canary。

模型在集合外请求场景中均返回不在 grant 内的工具选择。为避免工具名称承载额外语义，本文仅按 grant 集合成员关系判定；结果文件保留原始标识供复核。表 9 与确定性结果一致：G1R0 不能阻止任务后的集合外请求，G0R1 不能识别工具合法但来源不匹配的事件，G1R1 同时阻止两类未授权调用到达上游。该实验验证模型调用进入真实 MCP PEP 后的系统行为，不把工具选择比例解释为开放环境中的模型违规选择概率。

表 9　既有模型决策的配对执行路径

配置	正常上游到达	集合外请求到达	来源不匹配到达
G1R0	10/10	10/10	0/10
G0R1	10/10	0/10	10/10
G1R1	10/10	0/10	0/10

6.5.3　自有设施上的治理一致性与集合外请求回归

为检验独立部署后的执行链，本文在自有测试服务器上部署使用独立数据库和工作区的 E2AG 实例，并连接自有工具端点。允许路径使用公开测试仓库的真实 GitHub push，串联 Connector、任务容器、ToolGrant、MCP PEP 与外部工具。拒绝路径使用该仓库的另一条真实事件，由确定性测试程序发起固定的集合外工具请求，对应引言示例中“获准任务选择集合外工具”的反事实分支。

表 10　公开仓库–自有设施治理一致性配对回归

阶段	观测结果
允许路径正对照	匿名化提交摘要；任务 completed；5 次获准调用抵达自有工具端点；grant 终态 revoked
固定集合外请求负对照	grant 仅含 4 个评审工具；集合外请求返回 403 与 `MCP_TOOL_NOT_GRANTED`
未触达上游	拒绝审计的 `upstream_status=null`；canary 仍为 `effective=0`，记录数不变
因果与完整性	负对照 11 项审计链校验有效，包含 grant、`tool_call/deny`、revoke 与任务完成

该配对结果验证了公开仓库–测试服务器–任务运行时–PEP–工具端点的执行链：集合内调用到达上游，固定的集合外调用由 PEP 本地拒绝，上游状态保持不变。允许与拒绝记录共同核对了部署环境中的能力授权和运行时执行行为。

6.5.4　外部原实现与细粒度约束的配对验证

工具允许集合只约束工具选择，工具内部的资源和参数仍需单独授权。为检验参数约束，六个共同用例采用同一工具回执函数及业务规范：资源标识必须与获准对象相同，行数必须为 1 至 100 的整数，字段集合封闭且不进行类型转换。AgentSpec 以独立业务谓词表达规范，E2AG 以能力中的参数快照表达规范；工具端对实际执行的请求追加回执。表 11 比较仅检查工具集合、同时检查工具与参数的两种 E2AG 配置，以及 AgentSpec。

表 11　共同参数规范下的实际工具调用次数

场景	工具粒度 E2AG	AgentSpec	含参数约束 E2AG
合法参数	1	1	1
集合外工具	0	0	0
资源标识不匹配	1	0	0
行数超过上限	1	0	0
行数类型错误	1	0	0
行数恰好为上限	1	1	1

仅检查工具集合的 E2AG 配置在三项参数违规下仍执行工具；同时检查工具与参数的 E2AG 配置与 AgentSpec 均拒绝这些请求，并保留合法调用。另九项类型、边界和反向政策诊断也均符合预期。这表明允许工具集合需要与调用参数约束共同实施，而参数快照与运行时谓词能够在上述共同规范下实现一致的授权行为。

对完整上下文 OPA，五个准入与审批共同用例及一项硬拒绝优先控制在两个配置下形成 12 条记录，均符合预期：等待审批时不创建任务，批准后才恢复，重复提交不能再次创建任务，硬约束违规不因预置同意答复而获得放行。引擎不可用与策略拒绝分开处理，两个调用路径故障注入均返回 503、留下已鉴别请求的错误审计且没有工具回执。对 Agent libOS，正常、到期和原生撤销三项配对及到期前控制共八次执行结果也一致；所用原生 MCP 路径默认未记录拒绝审计，libOS 授权接口则支持通过配置启用审计。因此，该差异反映的是所测入口的配置方式。共同场景中各方法的执行结果一致，说明生命周期检查可由不同能力实现承担；跨层集成仍需确定事件、任务及审计记录的关联方式。

6.5.5 历史业务输入上的读写验证

为使调用检查接触实际业务对象，使用公开历史事件对应的两个 VS Code 开发对象构造确定性任务：issue #337178 的受理摘要和 PR #337123 的路径级变更清单。前者从 6016 字节历史对象中计算字段统计及受理队列；后者读取 7278098 字节的历史 Git 树快照包，从共同祖先至 head 计算文件新增、删除和修改。两者均经真实回环 HTTP 读取快照并写入隔离报告，不执行公开仓库源码。授权规则只绑定对象、版本和报告类型及长度，不预置正确报告或其摘要；独立检查器不导入报告生成函数，而从历史数据另行计算答案。

表 12　两类开发工作流中的共同执行结果

场景	任务	读取	写入	内容核验
正常业务	1	1	1	正确
读取对象或版本不匹配	1	0	0	无产物
写入对象不匹配	1	1	0	无产物
权限合法但报告内容错误	1	1	1	判错
事件类型未声明	0	0	0	无产物

原生 E2AG、E2AG-with-OPA 和共享 DiOS 准入宿主中的 AgentSpec 均得到表 12 的逐场景计数。issue 另含超长、错误类型和 Unicode 诊断控制，总计九场景、27 次运行；PR 为六场景、18 次运行。无门控控制确认不匹配请求能在同一工具执行，因而治理拒绝不是工具端预先拒绝造成的。权限合法但内容错误的报告仍能写入且被独立检查器判错，明确区分了授权正确性与业务内容正确性。

上述 45 次运行用于两个开发对象上的功能验证。工作流由固定执行器完成读写，原生任务记录保持 submitted，因此以工具端读写记录和报告内容衡量完成情况。比较中的 OPA 替换策略判定器，AgentSpec 共享 DiOS 事件准入宿主。为进一步检验相同规则在其他项目输入上的行为，下一组实验使用独立采样的历史对象。

6.5.6 跨项目历史事件上的配对比较

从 GH Archive 的公开事件小时档案 [28] 中提取 2026 年 9 月 22 日至 25 日的事件，时间采用 UTC、区间左闭右开。实验预先选择 ArcadeDB 的 issue 受理与 PR 变更清单、Kibana 的 issue 受理三个任务层，分别对应数据库项目与数据分析项目的协作输入。在历史内容获取和方法运行前，按 opened 动作筛选，排除已用于探索的事件、对象及提交，按对象保留最早事件并消除重复 head，再以固定种子哈希在每层选择至多 10 个对象。72 小时档案得到 11 个合格对象，三个任务层分别为 4、2、5 个，历史内容均可取得。Kibana PR 在预先可行性检查中未通过响应大小限制，故未列入采样范围。

各对象沿用两类开发工作流的规则：允许读取绑定对象的指定版本，并将报告写入对应结果对象；issue 和 PR 报告分别限制为 2048 和 65536 个 Unicode 码点。每个对象构造正常、读取对象不匹配、读取版本不匹配和写入对象不匹配四个场景，在 E2AG、E2AG-with-OPA 和共享准入宿主的 AgentSpec 中配对执行，共 132 条主运行。工具端为不匹配对象和版本设置可执行的测试别名，并以 33 项无门控请求验证其可达性。三个任务层的首个对象另设未声明事件与权限合法但内容错误两项诊断，共 18 条辅助运行。

核验程序由研究团队独立于报告生成函数实现，直接从冻结快照重算业务结果，并核对工具请求日志、落盘报告、任务与能力关联及审计链。issue 核验逐项计算字段统计和受理条件，PR 核验通过 SQL 集合连接生成变更清单，同时重建并检查 Git 树对象哈希。核验程序在正式执行前通过已知答案和错误报告测试。实验政策由研究团队制定，历史事件来自公共仓库；输入经标准化后在本地回环 HTTP 工具服务中回放。

表 13　11 个跨项目对象上的配对执行结果

配置	正常报告正确	越权读取到达	越权写入到达
E2AG	11/11	0/22	0/11
E2AG-with-OPA	11/11	0/22	0/11
AgentSpec（共享准入）	11/11	0/22	0/11

表 13 显示，三种配置均完成 11 个正常对象的读写且报告正确；读取对象或版本不匹配时，调用在读取前终止；写入对象不匹配时，合法读取完成而写入被拒绝。33 项无门控请求均可在工具端执行，说明拒绝由授权检查产生。辅助诊断中，三种配置均在未声明事件创建任务前拒绝；权限合法但内容错误的 9 份报告均被写入，并由业务核验器判错。全部 150 条主运行和辅助运行的工具记录、任务关联与审计链通过核对，未出现执行错误。

三种配置在不同项目对象上保持一致的授权结果：任务能力快照、完整上下文策略和运行时谓词均能实施相同的对象与版本约束。内容错误诊断进一步区分了授权检查与业务核验的职责，前者控制请求是否有权执行，后者判断产物是否符合业务规范。这组结果与内部消融相互补充：消融揭示两个执行位置各自保护的动作，跨项目比较检验共享授权规范在独立业务输入上的实施情况。

6.6　RQ3：证据定位与并发状态

6.6.1　故障定位与链验证

对契约拒绝、策略拒绝、审批过期、任务能力过期和 MCP 工具拒绝各注入 20 次。表 14 显示 100 条 trace 均保持统一标识、包含到达终态所需阶段，并把首个治理失败定位到注入位置。定位依据是显式 stage/outcome，阶段完整性依据是预期阶段集合覆盖，而非事后从文本日志猜测根因。该组检验阶段定位与记录完整性；对象绑定和授权前驱由第 6.6.3 节的关系反例进一步检查。

表 14　故障注入的治理阶段定位与证据验证

故障阶段	轨迹数	正确定位	链有效	阶段完整
契约拒绝	20	20	20	20
目标策略拒绝	20	20	20	20
审批过期	20	20	20	20
任务能力过期	20	20	20	20
MCP 工具拒绝	20	20	20	20

对内容修改、换序、trace 替换、前驱哈希替换、插入和中间删除的链内篡改，验证器均拒绝，与式 (9) 的校验条件一致。阶段记录支持失败位置定位，哈希关联支持已取得记录的链内完整性核对；尾截断与外部新鲜性问题在第 7.3 节讨论。

6.6.2　并发状态不变量

采用先查后插的去重配置在 8 并发的 100 轮实验中均产生重复 EventLog，共创建 588 条记录。采用唯一去重声明和审批条件更新的配置在表 15 的四组实验中均满足单日志或单终态不变量。重复请求共享同一去重声明，审批竞争通过条件更新选定唯一终态；这两项状态操作补充了策略判定，使重放和并发决定保持一致。

6.6.3　授权关系与调用结果状态核查

为区别哈希完整性与授权关系正确性，构造 19 项语义反例，包括任务或能力缺失、调用归属错误、意图重复、响应先于意图，以及结果状态与响应记录不一致。每次修改后重新计算整条哈希链，使链校验仍然通过；关系验证器在 19 项中均返回预期的错误类别。正常调用和结果未知的合法轨迹则保留各自状态。这些反例的哈希链保持有效，而对象关系不满足授权要求，表明关系核查能够发现仅靠哈希校验遗漏的错误。

对三份真实 HTTP 故障执行数据库进行只读证据核查，分别涉及意图提交失败、读取结果提交失败和写入结果提交失败。三个事件合计含三个任务、三个能力和三个调用意图，其中一个为 response_received、两个为 outcome_unknown；意图提交失败的事件没有生成

表 15　唯一去重与审批条件更新的并发状态检验

场景	并发	请求数	成功创建/终态	不变量违规
event replay	8	800	100	0
event replay	32	3200	100	0
approve/reject	8	800	100	0
approve/reject	32	3200	100	0

调用意图。快照均通过关系核查，检查前后数据库哈希不变。上述三份轨迹用于核查故障后的记录语义：结果未知表示本地证据无法确定上游结果，不等同于拒绝。

6.7　治理开销测量

6.7.1　同路径对照与测量方法

本组测量回答启用 E2AG 增加多少处理成本，以及正常业务完成情况如何变化。复用第 6.5.5 节的 issue 受理和 PR 变更清单两个开发工作流，固定执行器完成一次快照读取与一次报告写入，不调用模型。直通对照 G0 与完整治理 G2 使用同一原生 MCP 代理、连接复用、工具服务及 PostgreSQL 后端，均保存事件与任务记录。G0 以实验路由句柄转发，不作授权判定或写入治理证据；G2 保留来源准入、能力签发、调用检查、意图与响应持久化。两组均关闭事件去重，使每次到达创建独立任务。该设计保持公共业务路径一致，测量整套治理机制的增量。

实验运行于 4 逻辑 CPU、约 7.25 GiB 内存的共享测试服务器，使用 Python 3.12.14 与 PostgreSQL 16.13。应用、固定执行器和工具容器合计限制为 1.5 CPU、1536 MiB，数据库容器限制为 0.5 CPU、512 MiB，并启用同步提交。并发客户端数分别为 1 和 8，每个客户端完成一项业务后再发起下一项。每个工作流与并发组合执行 5 轮 G0/G2 配对，按固定种子改变先后顺序，共 40 次配置运行；每次使用独立数据库，预热 30 秒、测量 180 秒，预热请求排空后才开始测量。

客户端记录从事件发起至报告写入回执的业务延迟；服务端用同进程单调时钟记录准入、签发及调用前后处理等阶段。测量窗口截止后停止发起请求，在途完成数单列，不并入窗口内完成速率。延迟涵盖测量期发起且最终完成的请求，业务内容与持久记录在运行后核验。统计先在每轮计算 p50、p95 及窗口内完成速率，再取 5 轮的中位数；治理增量则先计算每对差值，再汇总其中位数与范围。计时检查在并发 1 下另作每工作流 5 对记录开关短测，issue 的 p50 变化为 -0.18% 至 $+0.46\%$，PR 为 -4.17% 至 $+0.95\%$，未观察到一致的延迟增加。

6.7.2　业务延迟与阶段开销

表 16 同时给出绝对性能与配对增量，测量期 103389 项业务均完成且报告正确。上半表的请求数为 5 轮合计，其余指标为各轮统计量的中位数；下半表先计算每轮 G2 相对 G0 的 p50 差值及相对差值，再汇总 5 轮中位数与最小–最大范围，范围不是置信区间。单并发下，两类业务增加的中位延迟均约 28 ms；并发 8 时均约 113–115 ms。相似的绝对增量在短业务中占比更大：issue 并发 8 的相对增量约 78%，其完成速率由 57.24 降至 31.05 次/s。对于这类短时、高频任务，治理开销已明显影响响应与处理能力，部署时需将其纳入延迟和容量预算。

表 17 进一步区分准入路径及每次工具调用的处理阶段。准入时间包括创建任务和准备调用能力；调用前处理从请求进入代理计至发送上游，调用后处理从取得上游响应计至生成代理响应。各阶段均列各轮 p50 的中位数，包含等待；嵌套阶段和不同阶段的分位数不直接相加。

并发 1 下，issue 准入路径从 10.40 增至 17.31 ms，每次调用前处理从 1.99 增至 6.55 ms，调用后处理从 0.15 增至 5.98 ms。完整治理中的契约、目标策略和参数判定 p50 均不足 0.05 ms，而能力签发、调用意图和响应记录分别为 4.16、3.83 和 5.84 ms，说明本组成本主要集中于状态访问与证据持久化路径，而非纯规则判定。PR 的响应记录为 11.34 ms，高于 issue 的 5.84 ms；该路径还包括响应摘要计算，因此响应内容也会影响治理成本。

并发 8 下，完整治理的 issue 准入、调用前后处理均增长，而工具往返仍接近直通对照，增量主要出现在宿主处理路径。PR 在两配置中的端到端 p50 均超过 2.4 秒，其 CPU 配额计数也均记录到节流；业务计算和资源竞争占据了较大的基准时间，使相对治理增量较小。因此，PR 约 5% 的相对增量与 issue 约 78% 的相对增量应结合各自的基准延迟理解，不能单独用于比较治理机制的效率。表中完成速率对应固定资源预算和闭环并发负载。

表 16　同路径对照的端到端结果及配对增量

业务	并发数	配置	p50/ms	p95/ms	完成速率/(次/s)	请求数
issue	1	G0	116.09	124.06	8.48	7638
issue	1	G2	144.35	152.26	7.07	6376
issue	8	G0	144.30	182.80	57.24	51581
issue	8	G2	256.96	319.80	31.05	27924
PR	1	G0	419.84	440.23	2.38	2141
PR	1	G2	446.40	466.61	2.27	2042
PR	8	G0	2488.48	3328.48	3.18	2900
PR	8	G2	2611.48	3254.11	3.05	2787

业务	并发数	增量/ms	轮次范围/ms	增量/%	轮次范围/%
issue	1	28.26	28.09–28.38	24.34	24.20–24.45
issue	8	112.50	110.21–117.83	77.88	76.29–81.66
PR	1	27.81	22.32–31.66	6.62	5.26–7.61
PR	8	114.72	74.33–149.38	4.62	2.99–6.03

增量按每轮 G2 相对 G0 的 p50 配对计算，列出 5 轮中位数和最小–最大范围。

表 17　准入及工具调用的分阶段墙钟时间（ms）

业务	并发数	配置	准入路径	调用前处理	工具往返	调用后处理
issue	1	G0	10.40	1.99	47.49	0.15
issue	1	G2	17.31	6.55	47.60	5.98
issue	8	G0	20.93	4.85	49.76	0.69
issue	8	G2	65.14	25.05	50.43	18.37
PR	1	G0	10.47	2.12	58.26	0.16
PR	1	G2	17.38	7.03	58.07	11.49
PR	8	G0	21.82	5.11	117.11	0.23
PR	8	G2	52.56	22.36	114.43	12.67

6.8　异构接入、工具传输与持久化验证

前述实验分别检验业务输入变化和治理机制分工；本组进一步考察交付、传输及持久化方式变化后，相同授权约束能否保持。接入选择直接 HTTP 请求、AMQP 代理交付和非网络文件发布，分别涉及请求鉴别、消息确认与重投递、文件完成发布与重复扫描；工具侧比较远程 HTTP 与本地 stdio 管道，状态侧比较嵌入式与服务式事务存储。这些方式覆盖了直接请求、消息代理和文件交换三种交付机制。测试围绕来源绑定、事件转换、事件身份、交付恢复、强制检查和证据关联六项契约组织，考察相同授权约束在这些机制下的实施情况。

6.8.1　入口鉴别与多目标准入

来源鉴别与多目标准入通过八项本地集成测试检查。GitHub、GitLab、Gitea 经原生接入处理函数及标准化器，在有效凭据下接入推送，错误凭据在 EventLog 创建前返回 403；GitHub/Gitea 签名可发现请求体变化，而 GitLab 共享令牌本身不绑定请求体。空密钥可进入开发旁路，说明部署时必须关闭该旁路并配置有效凭据，才能满足来源鉴别前提 A1。多目标测试覆盖全部目标允许、一目标违反约束、

整体等待审批和批准后恢复四种状态，重复提交批准请求未创建额外任务。该组使用隔离数据库检查入口与状态处理，不启动任务容器；下节进一步检查实际传输与持久化替换后的执行结果。

6.8.2 跨机制配置与执行结果

表 18 给出五个单因素主配置与一个组合配置。各主配置执行正常、来源拒绝、待审批、批准后执行、未授权工具、越界资源、撤销和过期八个场景，并实际重复交付。输入及政策相同：将指定文本转为大写，只允许写入绑定报告对象。适配器接入原生 DiOS 订阅、调度、审批和能力过程；固定执行器替代任务容器启动，经进程内异步服务器网关接口（Asynchronous Server Gateway Interface，ASGI）调用原生 PEP，代理至工具使用实际网络或进程管道。每场景使用独立数据库；AMQP 代理和服务式数据库分别为 RabbitMQ 3.13.7 与 PostgreSQL 16.13。工作流不调用模型，其配置–场景运行与第 6.5.6 节的独立历史业务对象分开统计。

表 18 异构配置与治理一致性结果

配置	事件接入	代理至工具	权威状态	符合预期/场景数
R0	HTTP Webhook	MCP HTTP	SQLite	8/8
R1	AMQP 消费	MCP HTTP	SQLite	8/8
R2	文件投递箱	MCP HTTP	SQLite	8/8
R3	HTTP Webhook	MCP stdio	SQLite	8/8
R4	HTTP Webhook	MCP HTTP	PostgreSQL	8/8
R5	AMQP 消费	MCP stdio	PostgreSQL	2/2（组合）

R0–R4 的逐场景结果一致：正常与批准后业务各完成一次正确写入；来源拒绝和待审批不创建任务；四类调用违规均在 PEP 被拒绝，工具端未收到相应请求。重复交付没有增加任务或产物。组合配置的第一项在待审批事件持久交付后、消息确认前终止消费者，实际重投递并恢复后只批准和执行一次；第二项在撤销提交并关闭原管道后请求重新初始化和工具调用，两者均被拒绝且未启动新工具进程。42 项配置–场景运行共包含 11 次预期工具写入，其余场景按规则拒绝或保持等待。

由研究团队与被测流程分开实现的检查器直接读取数据库、实际传输记录和工具产物，核对事件、任务、审批、能力、调用意图与响应的关联及链内哈希。正确轨迹通过；另以破坏能力关联、错误权限快照、重算哈希后删除调用意图和额外工具写入检验检查器，均发现预期错误。接口故障检查还覆盖签名与字段错误、同标识内容冲突、确认或响应丢失，以及意图和结果持久化故障。任务创建中途终止的一项保留未决状态，不自动重放或计为完成。服务式后端的状态操作需要满足三项约束：审计字段支持相等比较，审计追加在事务内读取已提交的链，过期处理以待审批状态为更新条件。缺少这些约束时，分别会出现 JSON 相等比较不兼容、旧会话覆盖审计链及过期处理覆盖审批终态的问题。这说明后端替换不仅需要接通接口，还需保持状态迁移和证据追加的语义。

该结果支持所测适配契约下的治理一致性：交付形式改变后，规范事件仍携带相同授权字段；工具转发仍位于能力检查与意图持久化之后；事务后端仍维护单次状态迁移和对象关联。其适用前提包括可信入口配置、不可变文件发布、已确认旧消费者停止后的恢复，以及受控工具出口。实验覆盖上述接入、传输和状态机制，但固定工作流与单一 Agent OS 仍限制其对其他运行时的外推。

7 讨论

7.1 方法适用范围

E2AG 适用于外部事件经统一调度入口创建任务、外部工具调用经可中介出口执行的 Agent OS。其治理目标是使调用不超出获准来源、目标和工具集合，而不是判定自然语言意图类别。仅配置工具集合时，合法工具名内部的参数违规无法区分；参数约束 K 检查声明的资源标识、字段类型和取值范围，但仍不能保证自然语言内容正确或约束跨调用的数据流。进一步结合数据流机制不改变本文事件–任务–工具的授权层定义。

命题 1 只适用于满足 A1–A3 的事件驱动任务路径。表 3 区分已中介、禁用和未覆盖出口：管理员允许清单中的 stdio 服务由能力代理启动并管理，未启用的 stdio 及 SSE 配置仍不下发。该控制不限制运行时自行建立网络连接。直接 A2A 入口、服务模式、Skill 和任意本地进程仍需访问隔离或额外拦截。移植到其他 Agent OS 时，应识别对应入口和出口、隔离可直连凭据，并检验条件更新、恢复与证据关联等契约；仅移植策略函数不足以沿用安全结论。

细粒度机制可在此基础上组合：资源解析器先将路径或服务别名转换为稳定对象标识，能力快照保存该作用域，调用 PEP 对实际对象和参数实施 K；跨步骤敏感数据流则由 CaMeL 类机制传播标签并限制接收端。原型已实现标识、类型、范围和长度检查，但未实现上述完整资源解析和数据流传播。二者分别处理调用权限与数据传播，可在同一调用出口组合实施。

7.2 有效性威胁

7.2.1 构念效度

冻结矩阵按治理规则构造，预期标签与规则定义紧密相关。四名未参与构造的复核者对治理层和预期决策完全一致，但对 4 个审批敏感事件的攻击/正常类别存在分歧。本文因此采用放行、拒绝或审批作为主要标签，并单独报告类别一致性。该设计有助于检验规则实施及机制分工；对自然流量中违规事件的发生率与检出率，仍需使用相应分布的样本。

7.2.2 内部有效性

既有模型结果采用冻结提示与工具选择，使相同决策能够在不同配置下配对复用，隔离 PEP 布置对上游调用可达性的影响。历史业务比较固定输入、政策和工具服务，并通过原始记录及独立实现的内容核验减少输出自证风险。不过，核验器仍由同一研究团队编写，可能与实现共享对规范的误解；已知答案、错误报告和无门控控制分别检查其计算、判错与观测能力。这组实验衡量指定规则下的授权一致性，开放环境中的模型选择概率需要另行采样。

性能测量使用共享服务器及固定容器配额。公共路径一致、配对顺序变化和 5 轮重复减少配置间混杂，但同宿主调度、数据库等待及客户端业务计算仍会影响端到端时间。分阶段计时用于辨别这些成本，结果按轮统计并保留变化范围；PR 并发 8 的资源节流与测量条件一并报告。该测量支持所测业务及资源配置下的治理成本判断，不将成功请求数量视为独立工作负载规模。

7.2.3 外部有效性

实验基于 DiOS，模型回放涉及一个模型。为分别观察输入变化与环境变化的影响，跨项目实验采用公共历史事件，异构实验在相同授权规则下替换接入、传输和存储方式，并核对实际接口与持久记录。这些控制使观测结果能够对应到所替换的因素，但历史项目经过有目的选择、独立对象较少，异构工作流与适配器也由研究团队构建。另一 Agent OS 的任务入口、工具出口和状态管理需要重新接入，因此本文将实验结论限定于 DiOS 及所测适配契约；跨宿主迁移尚需单独验证。

7.3 审计证据的保证范围

在可信记录端与存储的前提下，哈希链检查已取得记录的内容和顺序，关系验证器进一步检查任务、能力、审批与调用记录的绑定和必要前驱。19 项语义反例说明这两类义务不能互相替代；三份持久轨迹的复核也区分了结果未知与响应已观察状态。由于原始参数、历史策略和外部执行收据并不齐备，这些核查仍不覆盖 I4 的全部语义，也不构成跨组织不可否认性。哈希链不能证明数据库未被整体回滚或尾截断；外部透明日志或周期性 Merkle 根锚定可以强化新鲜性与完整性，但不改变 I1–I3 的授权语义。

8 总结

本文针对事件驱动 Agent OS 中事件来源、任务创建和工具调用跨越不同信任域的问题，提出跨层能力治理方法 E2AG。该方法用来源–类型契约建立事件声明权，用目标策略和审批控制任务创建，用任务作用域能力与调用时 PEP 约束实际工具及其参数，并通过统一执行证据连接各阶段对象。状态迁移模型明确了受控路径上的授权前提，消融揭示了准入与调用检查的互补作用，跨项目历史事件比较验证了正常业务保持和指定越权请求的阻断，故障注入与并发实验检验了证据关系和状态单调性。异构配置进一步表明，在所测适配契约下，更换事件交付、工具传输和事务后端保持了相同授权结果及证据关联。

完整上下文 OPA 和共享准入宿主的 AgentSpec 在共同业务场景中获得一致结果，Agent libOS 在共同生命周期场景中也实施了相同约束。这些结果说明，跨层对象组织与具体判定机制可以分离，现有策略及能力原语可承担相应授权检查。同路径性能测量量化了该治理结构的实施成本：在保持正常业务完成的同时，状态访问与证据持久化增加了处理延迟，其相对影响随业务计算量和并发条件变化。后续研究将围绕不同运行时出口的治理接入、资源规范化和跨调用数据流约束展开。

References

[1] Mei K, Zhu X, Xu W, et al. AIOS: LLM agent operating system. In: Proc. of the 2nd Conf. on Language Modeling. Montreal: OpenReview, 2025.

[2] Wang L, Ma C, Feng X, et al. A survey on large language model based autonomous agents. Frontiers of Computer Science, 2024, 18(6):

186345. [doi: 10.1007/s11704-024-40231-1]

[3] Huang HY, Li SL, Lan TW, et al. A survey on the safety of large language model: Classification, evaluation, attribution, mitigation and prospect. CAAI Transactions on Intelligent Systems, 2025, 20(1): 2–32 (in Chinese with English abstract). [doi: 10.11992/tis.202401006]

[4] Mu YY, Chen HX, Li HW. Advances in security and privacy-preserving techniques for large language models. Journal of Cybersecurity, 2024, 2(1): 40–49 (in Chinese with English abstract). [doi: 10.20172/j.issn.2097-3136.240103]

[5] Zhang X, Li CZ, Xu N, Zhang LT. Security challenges and response mechanisms for trustworthy large language model agents. Information and Communications Technology and Policy, 2025, 51(1): 33–37 (in Chinese with English abstract). [doi: 10.12267/j.issn.2096-5931.2025.01.005]

[6] Abdelnabi S, Greshake K, Mishra S, et al. Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In: Proc. of the 16th ACM Workshop on Artificial Intelligence and Security. New York: ACM, 2023. 79–90. [doi: 10.1145/3605764.3623985]

[7] Zhan Q, Liang Z, Ying Z, Kang D. InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents. In: Proc. of the Findings of ACL 2024. Bangkok: Association for Computational Linguistics, 2024. 10471–10506. [doi: 10.18653/v1/2024.findings-acl.624]

[8] Debenedetti E, Zhang J, Balunović M, et al. AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. Advances in Neural Information Processing Systems, 2024, 37: 82895–82920. [doi: 10.52202/079017-2636]

[9] Ruan Y, Dong H, Wang A, et al. Identifying the risks of LM agents with an LM-emulated sandbox. In: Proc. of the 12th Int'l Conf. on Learning Representations. Vienna: OpenReview, 2024.

[10] Wang H, Poskitt CM, Sun J. AgentSpec: Customizable runtime enforcement for safe and reliable LLM agents. In: Proc. of the 48th IEEE/ACM Int'l Conf. on Software Engineering. New York: ACM, 2026. 12 pages. [doi: 10.1145/3744916.3764546]

[11] Debenedetti E, Shumailov I, Fan T, et al. Defeating prompt injections by design. In: Proc. of the IEEE Conf. on Secure and Trustworthy Machine Learning, 2026.

[12] Uchibeke U. Before the Tool Call: Deterministic Pre-Action Authorization for Autonomous AI Agents. arXiv:2603.20953, 2026.

[13] Ravindran A, Deochake S. ToolGuardian: Declarative Security for AI Agent-Tool Interactions. arXiv:2607.21835, 2026.

[14] El Helou M, Ryder B, Troiani C, et al. Hybrid Inspection and Task-Based Access Control in Zero-Trust Agentic AI. arXiv:2605.02682, 2026.

[15] Zhang YQ. Agent libOS: A Runtime Substrate for Capability-Controlled Self-Evolving LLM Agents. arXiv:2606.03895v2, 2026.

[16] Cloud Native Computing Foundation. CloudEvents Specification, Version 1.0.2, 2022. `https://github.com/cloudevents/spec/tree/ce@v1.0.2`. [2026-08-14]

[17] A2A Protocol Project. Agent2Agent Protocol, Version 1.0.0, 2026. `https://a2a-protocol.org/v1.0.0/`. [2026-08-17]

[18] Model Context Protocol Contributors. Model Context Protocol Specification, Revision 2025-06-18. `https://modelcontextprotocol.io/specification/2025-06-18/`. [2026-08-14]

[19] Open Policy Agent. OPA documentation; Release v1.17.0. `https://www.openpolicyagent.org/docs`; `https://github.com/open-policy-agent/opa/releases/tag/v1.17.0`. [2026-08-17]

[20] Anderson JP. Computer Security Technology Planning Study. Technical Report ESD-TR-73-51, 1972.

[21] Saltzer JH, Schroeder MD. The protection of information in computer systems. Proc. of the IEEE, 1975, 63(9): 1278–1308. [doi: 10.1109/PROC.1975.9939]

[22] Dennis JB, Van Horn EC. Programming semantics for multiprogrammed computations. Communications of the ACM, 1966, 9(3): 143–155. [doi: 10.1145/365230.365252]

[23] Birgisson A, Politz JG, Erlingsson Ú, et al. Macaroons: Cookies with contextual caveats for decentralized authorization in the cloud. In: Proc. of the Network and Distributed System Security Symp. San Diego: Internet Society, 2014.

[24] Xu JL, Wang SL, Li LM, et al. Formal verification of capability-based access control in operating system kernel. Journal of Software, 2025, 36(8): 3570–3586 (in Chinese with English abstract). [doi: 10.13328/j.cnki.jos.007351]

[25] W3C. Trace Context. W3C Recommendation, 2021. `https://www.w3.org/TR/trace-context/`.

[26] Haber S, Stornetta WS. How to time-stamp a digital document. Journal of Cryptology, 1991, 3(2): 99–111. [doi: 10.1007/BF00196791]

[27] Torres-Arias S, Afzali H, Kuppusamy TK, et al. in-toto: Providing farm-to-table guarantees for bits and bytes. In: Proc. of the 28th USENIX Security Symp. Santa Clara: USENIX Association, 2019. 1393–1410.

[28] GH Archive. GH Archive: Public GitHub activity archives. https://www.gharchive.org/. [2026-10-07]